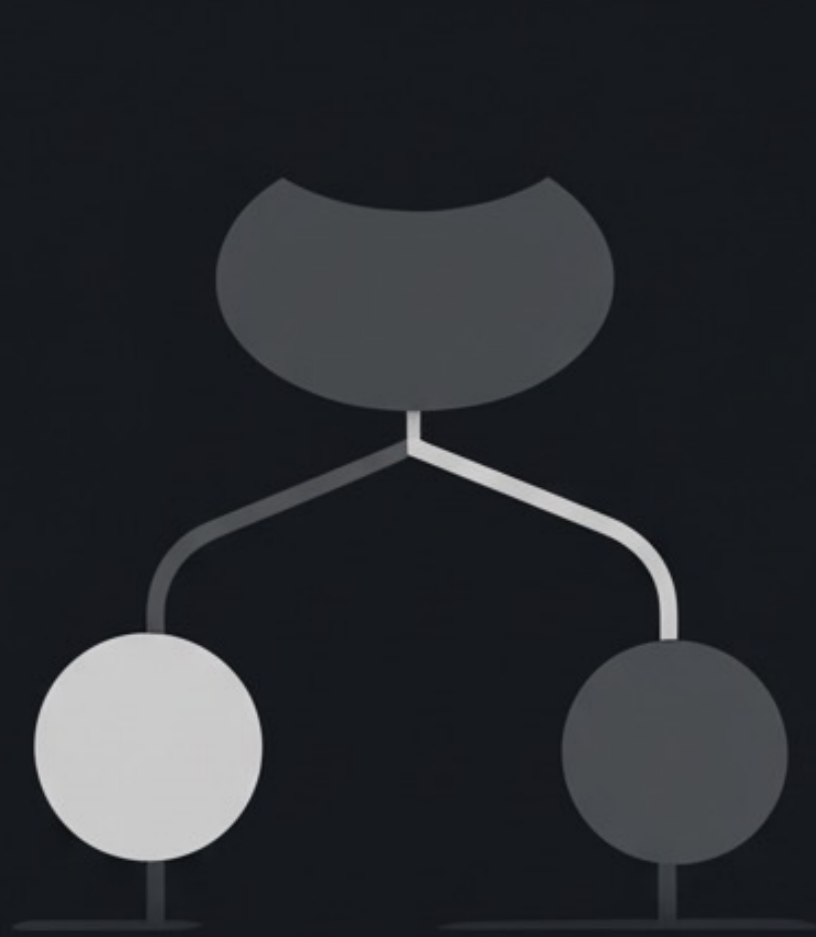


DATA STRUCTURES AND ALGORITHMS



QUEUES

C {...}



Queue ADT

Data:

- 1) Space for storing elements
- 2) Front $\rightarrow$ For deletion
- 3) Rear $\rightarrow$ For Insertion

Operations:

- 1) enqueue(x) $\rightarrow$ Inserting an element
- 2) dequeue() $\rightarrow$ deleting an element
- 3) isEmpty()
- 4) isFull()
- 5) first()
- 6) Last()

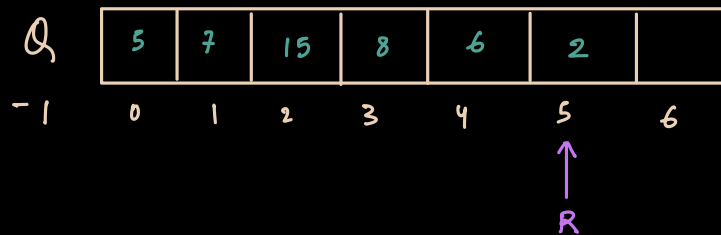
Queue Implementation using Array

- 1) Queue implementation using single pointer
- 2) Queue implementation using front and rear pointers.
- 3) Drawbacks of queue implementation using array.

1. Queue implementation using Single pointer

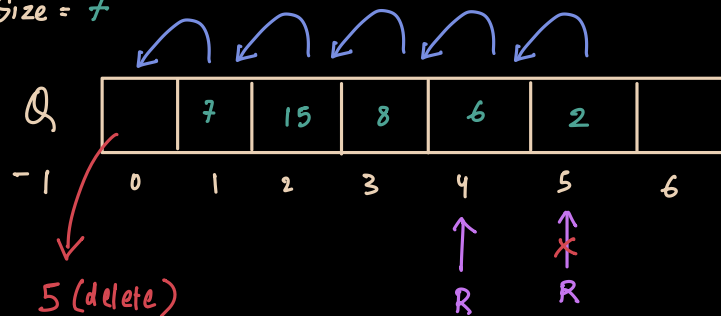
(a) Enqueue - $O(1)$

Size = 7



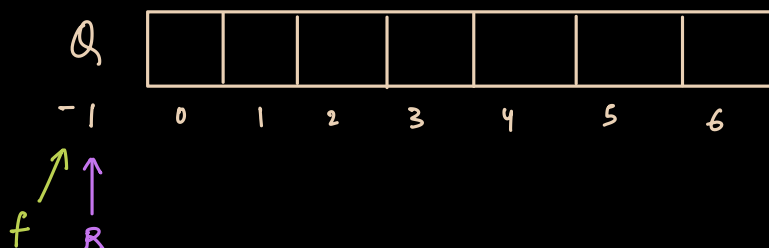
(b) Dequeue - $O(n)$ → disadvantage

Size = 7



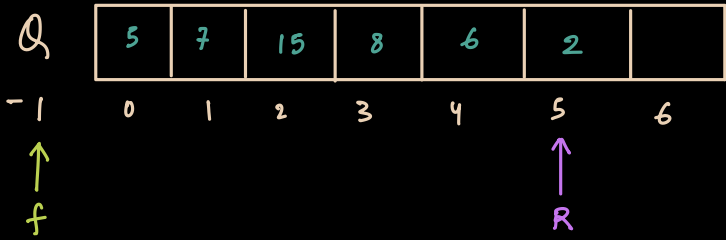
2. Queue implementation using front and rear pointers

Size = 7



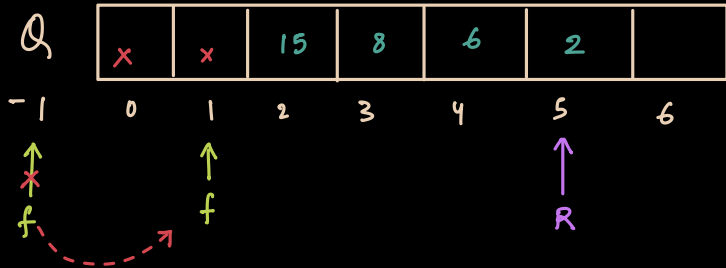
(a) Enqueue - $O(1)$

Size = 7



(b) Dequeue - $O(1)$

Size = 7



Note : Initially $\rightarrow \text{front} = \text{Rear} = -1$
Empty $\rightarrow \text{if (front == Rear)}$
Full $\rightarrow \text{if (Rear = size - 1)}$

Empty $\rightarrow$ if (front == Rear)

Full $\rightarrow$ if (Rear = size - 1)

CODE :

```
struct Queue {  
    int size ;  
    int front ;  
    int Rear ;  
    int *Q ;  
}  
  
void enqueue ( Queue *q , int x ) {  
    // is Full  
    if ( q → Rear == q → size - 1 )  
        printf ( " Queue is full " ) ;  
    else {  
        q → Rear ++ ;  
        q → Q [ q → Rear ] = x ;  
    }  
}  
  
void dequeue ( Queue *q ) {  
    int x = -1 ;  
    // is Empty  
    if ( q → Rear == q → front )  
        printf ( " Queue is Empty " ) ;  
    else {  
        q → front ++ ;  
        x = q → Q [ q → front ] ;  
    }  
    return x ;  
}
```

```
int main() {
```

```
    struct Queue q;
```

```
    printf("Enter size: ");
```

```
    scanf("%d", &q.size);
```

```
    q.front = q.Rear = -1;
```

```
    q.Q = (int *) malloc(q.size * sizeof(int));
```

```
}
```

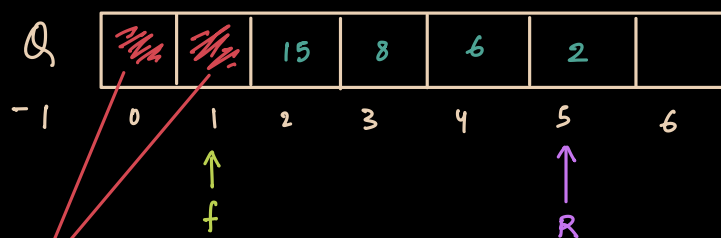
Size
front
Rear
Q

7
-1
-1
Q



3. Drawbacks of Queue using Array

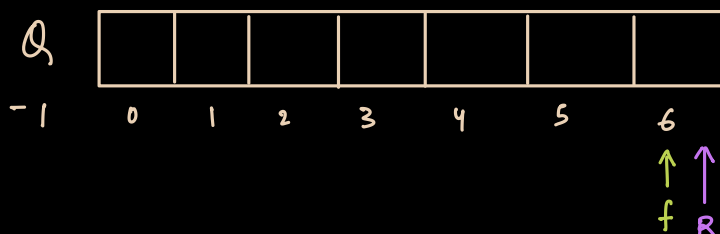
1) Cannot use the free space of deleted elements



These two free spaces cannot be utilised for insertion

2) Every location can only be used once.

3) A queue can simultaneously be empty as well as full.



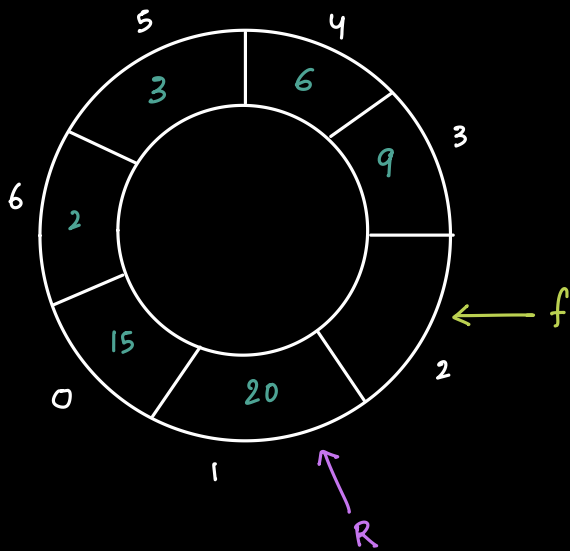
SOLUTIONS

- 1) Resetting pointers :
- a) If Queue is empty.
 - b) Re-initialise front and rear pointer to -1

But this method doesn't guarantee reusing of spaces.

- 2) Circular queue

Circular Queue



* Same for front pointer

$$\text{Rear} = (\text{Rear} + 1) \% \text{size} =$$

0	$(0+1) \% 7$	1
1	$(1+1) \% 7$	2
2	$(2+1) \% 7$	3
3	$(3+1) \% 7$	4
4	$(4+1) \% 7$	5
5	$(5+1) \% 7$	6
6	$(6+1) \% 7$	0
0		

o Enqueue $\rightarrow O(1)$

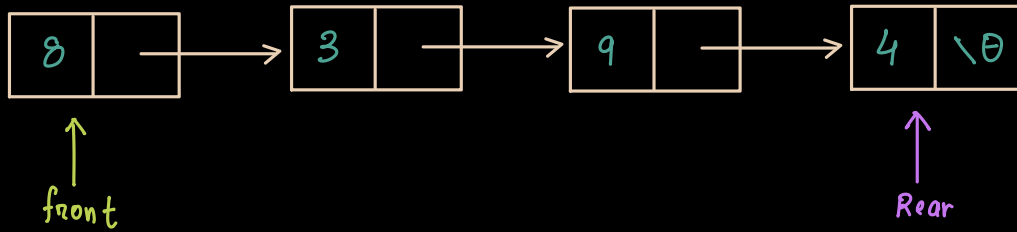
```

void enqueue ( struct Queue *q, int x ) {
    if ((q->Rear+1) % q->size == q->front)
        printf("Queue is full");
    else {
        q->Rear = (q->Rear+1) % q->size;
        q->Q[q->Rear] = x;
    }
}
    
```


$O(\text{dequeue}) \rightarrow O(1)$

```
int dequeue(struct Queue *q) {  
    int x = -1 ;  
    if ( q->front == q->Rear )  
        printf(" Queue is empty " ) ;  
    else {  
        q->front = ( q->front + 1 ) % q->size ;  
        x = q->Q[ q->front ] ;  
    }  
    return x ;  
}
```

Queue Implementation using Linked List



o is Empty

```
if( front == NULL )
```

// Any one of front or Rear is NULL

o is Full

```
Node *t = new Node ;
```

```
if( t == NULL )
```

o enqueue $\rightarrow O(1)$

```
void enqueue( int x ) {
```

```
    Node *t = new Node ;
```

```
    if( t == NULL )
```

```
        printf( " full " );
```

```
    else {
```

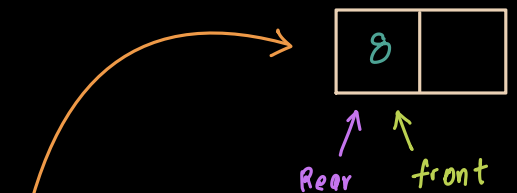
```
        t->data = x ;
```

```
        t->next = NULL ;
```

```
        if( front == NULL )
```

```
            front = Rear = t ;
```

// Extra case to check
if its the first node.



```

    }
    }
    else {
        Rear → next = t ;
        Rear = t ;
    }
}

```

O dequeue → $O(1)$

```

int dequeue( ) {
    int x = -1 ;
    Node *p ;
    if( front == NULL )
        printf( " empty " ) ;
    else {
        p = front ;
        front = front → next ;
        x = p → data ;
        free ( p ) ;
    }
    return x ;
}

```

DEQueue - Double Ended Queue

Queue

	Insert	Delete
front	X	✓
Rear	✓	X

DEQueue

	Insert	Delete
front	✓	✓
Rear	✓	✓

Types: (1) i/p restricted DEQueue

	Insert	Delete
front	X	✓
Rear	✓	✓

(2) o/p restricted DEQueue

	Insert	Delete
front	✓	✓
Rear	✓	X



ARRAY IMPLEMENTATION OF DEQueue



STEP 1: Define the Structure

We define a Deque struct that holds:

- A dynamically allocated array
- The front and rear indices
- The size of the array

```
c
typedef struct {
    int *arr;    // Pointer to dynamic array
    int front;
    int rear;
    int size;    // Size of the array
} Deque;
```



STEP 2: Create & Initialize the DEQueue

```
c
Deque* createDeque(int size) {
    Deque* dq = (Deque*)malloc(sizeof(Deque));
    dq->size = size;
    dq->arr = (int*)malloc(sizeof(int) * size);
    dq->front = -1;
    dq->rear = -1;
    return dq;
}
```



STEP 3: Check Full and Empty Conditions

Is Full?

```
c Copy
int isFull(Deque* dq) {
    return ((dq->front == 0 && dq->rear == dq->size - 1) ||
           (dq->rear == (dq->front - 1 + dq->size) % dq->size));
}
```

Is Empty?

```
c Copy
int isEmpty(Deque* dq) {
    return dq->front == -1;
}
```



STEP 4: Insert at Front

```
c
void insertFront(Deque* dq, int value) {
    if (isFull(dq)) {
        printf("Overflow! Cannot insert at front.\n");
        return;
    }

    if (isEmpty(dq)) {
        dq->front = dq->rear = 0;
    } else if (dq->front == 0) {
        dq->front = dq->size - 1;
    } else {
        dq->front--;
    }

    dq->arr[dq->front] = value;
}
```

STEP 5: Insert at Rear

c

Copy

```
void insertRear(Deque* dq, int value) {
    if (isFull(dq)) {
        printf("Overflow! Cannot insert at rear.\n");
        return;
    }

    if (isEmpty(dq)) {
        dq->front = dq->rear = 0;
    } else if (dq->rear == dq->size - 1 && dq->front != 0) {
        dq->rear = 0;
    } else {
        dq->rear++;
    }

    dq->arr[dq->rear] = value;
}
```

STEP 6: Delete from Front

c

Copy

```
void deleteFront(Deque* dq) {
    if (isEmpty(dq)) {
        printf("Underflow! Cannot delete from front.\n");
        return;
    }

    int value = dq->arr[dq->front];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else if (dq->front == dq->size - 1) {
        dq->front = 0;
    } else {
        dq->front++;
    }

    printf("Deleted from front: %d\n", value);
}
```

STEP 7: Delete from Rear

c

Copy

```
void deleteRear(Deque* dq) {
    if (isEmpty(dq)) {
        printf("Underflow! Cannot delete from rear.\n");
        return;
    }

    int value = dq->arr[dq->rear];

    if (dq->front == dq->rear) {
        dq->front = dq->rear = -1;
    } else if (dq->rear == 0) {
        dq->rear = dq->size - 1;
    } else {
        dq->rear--;
    }

    printf("Deleted from rear: %d\n", value);
}
```

🔗 STEP 8: Display (Optional but helpful for debugging)

```
c Copy

void displayDeque(Deque* dq) {
    if (isEmpty(dq)) {
        printf("Queue is empty.\n");
        return;
    }

    printf("Deque contents: ");
    int i = dq->front;
    while (1) {
        printf("%d ", dq->arr[i]);
        if (i == dq->rear) break;
        i = (i + 1) % dq->size;
    }
    printf("\n");
}
```

🔗 STEP 9: Test It

```
c Copy

int main() {
    Deque* dq = createDeque(5);

    insertRear(dq, 10);
    insertRear(dq, 20);
    insertFront(dq, 5);
    insertFront(dq, 1);

    displayDeque(dq);

    deleteRear(dq);
    deleteFront(dq);

    displayDeque(dq);

    free(dq->arr);
    free(dq);
    return 0;
}
```

Priority Queues

two methods of implementation :

1. limited set of priorities
2. Element priority

(1) limited set of priorities

Ex : Priorities = 3

Element	→	A	B	C	D	E	F	G	H	I	J
priority	→	1	1	2	3	2	1	2	3	2	2

here 1, 2, 3

←
Priority
inc.

For implementing this, we have to take 3 Queues.

Q ₁	A	B	F			
----------------	---	---	---	--	--	--

Q ₂	C	E	G	I	J	
----------------	---	---	---	---	---	--

Q ₃	D	H				
----------------	---	---	--	--	--	--

* For deletion, order must be from higher priority queue to lower priority queue.

ie ; Every element should be deleted first from Q₁ in FIFO fashion.

order of deletion : A, B, F, C, E, G,

2. Element priority

Elements $\rightarrow$ 6, 8, 3, 10, 15, 2, 9, 17, 5 (Smaller no.
higher priority)

Q

--	--	--	--	--	--	--	--	--

Two methods :

1. Insert in same order $\rightarrow O(1)$

Delete max priority by searching it. $\rightarrow$ Searching - $O(n)$
shifting - $O(n)$

$$O(2n) = O(n)$$

Q

6	8	3	10	15	2	9	17	5
---	---	---	----	----	---	---	----	---

delete max prior. (2) Q

6	8	3	10	15	9	17	5	
---	---	---	----	----	---	----	---	--

then (3) Q

6	8	10	15	9	17	5		
---	---	----	----	---	----	---	--	--

then (5) Q

6	8	10	15	9	17			
---	---	----	----	---	----	--	--	--

⋮

2. Insert in inc. order of priority $\rightarrow$ shift - $O(n)$

Delete last element of array $\rightarrow O(1)$

Q

6								
---	--	--	--	--	--	--	--	--

(8) Q

8	6							
---	---	--	--	--	--	--	--	--

 (move 6)

(3) Q

8	6	3						
---	---	---	--	--	--	--	--	--

(10) Q

10	8	6	3					
----	---	---	---	--	--	--	--	--

$\left(\begin{array}{l} \text{move } 3 \\ \text{" } 6 \\ \text{" } 8 \end{array} \right)$

(15) Q

15	10	8	6	3				
----	----	---	---	---	--	--	--	--

$\left(\begin{array}{l} \text{move } 3 \\ \text{" } 6 \\ \text{" } 8 \\ \text{" } 10 \end{array} \right)$

⋮

deletion $\rightarrow$ last element first

Q

15	10	8	6	x				
----	----	---	---	---	--	--	--	--

Q

15	10	8	x	x				
----	----	---	---	---	--	--	--	--

Q

15	10	x	x	x				
----	----	---	---	---	--	--	--	--

⋮

